Lecture II - Control Structures for Your Code

Programming with Python

Dr. Tobias Vlćek
Kühne Logistics University Hamburg - Fall 2026

Episode 2: The Curfew

A decree from the city

Overnight the city banned delivery after **22:00**. Your app can no longer just say yes to every order. Today the code starts making decisions, repeating work over every order, and tidying up Tobi's menu.

. . .

(Tobi's fix: "we just deliver yesterday's orders the next morning." A lawyer disagreed.)

🔥 Warm-up

Three questions from Episode 1. Commit. Hands up **before** the reveal.

Question 1

```
price = 8.90
print(f"{price * 2:.2f}")
```

a) `17.8` b) `17.80` c) `Error`

Answer 1

b): `:.2f` always prints two decimals. That's the whole receipt trick.

Question 2

```
print(type("9.99"))
```

a) `float` b) `int` c) `str`

Answer 2

c): quotes make it text, no matter how numeric it looks. (Tobi learned this the hard way.)

Question 3

```
print(7 // 2, 7 % 2)
```

a) `3.5 1` b) `3 1` c) `3 0.5`

Answer 3

b): `//` floors, `%` gives the remainder. Together: "how many fit, what's left."

Making Decisions

Asking a yes/no question

A **comparison** asks a question and answers with a **boolean**: `True` or `False`, the `bool` type you met last week:

```
delivery_hour = 21
print(delivery_hour < 22)    # before curfew?
print(delivery_hour == 22)   # exactly at curfew?
```

```
True
False
```

The comparison operators

Six operators, all returning `True` or `False`:

- `<` less than · `>` greater than
- `<=` at most · `>=` at least
- `==` equal · `!=` not equal

. . .

> ⚠ Warning
>
> `=` **assigns** a value; `==` **compares** two values. Mixing them up is the classic first-week bug.

Combining conditions

`and`, `or`, `not` glue booleans together:

```
is_weekday = True
before_curfew = 21 < 22
print(is_weekday and before_curfew)   # both must be True
```

```
True
```

. . .

`and` needs **both** sides true; `or` needs **at least one**; `not` flips it.

`if`: do something only when

An `if` runs its indented block **only** when the condition is `True`:

```
delivery_hour = 21
if delivery_hour < 22:
    print("Open: send the courier.")
```

```
Open: send the courier.
```

...

The **indentation** (4 spaces) is what marks the block. Python is strict about it.

`if` / `else`: two paths

`else` catches every case the `if` missed:

```
delivery_hour = 23
if delivery_hour < 22:
    print("Open: send the courier.")
else:
    print("Curfew: kitchen closed.")
```

```
Curfew: kitchen closed.
```

`if` / `elif` / `else`: a ladder

More than two cases? Stack `elif` branches. Reward loyal customers by tier:

```
past_orders = 8
if past_orders >= 20:
    tier = "Gold"
elif past_orders >= 5:
    tier = "Silver"
else:
    tier = "Bronze"
print(tier)
```

```
Silver
```

Predict: Tobi reorders the ladder

Tobi rewrote the tier ladder "to keep the common case on top". A customer with **25 past orders** walks in. Which tier do they get?

```
past_orders = 25
if past_orders >= 5:
    tier = "Silver"
elif past_orders >= 20:
    tier = "Gold"
else:
```

```
    tier = "Bronze"
print(tier)
```

a) `Gold` b) `Bronze` c) `Silver`

...

Predict first. Commit to an answer before the next slide.

Answer: the first true branch wins

c) **`Silver`**: Python checks branches top to bottom and takes the first `True` one. `25 >= 5` is already `True`, so the `Gold` branch is never even looked at. Order your ladder from strictest to loosest.

```
past_orders = 25
if past_orders >= 5:
    tier = "Silver"
elif past_orders >= 20:
    tier = "Gold"
else:
    tier = "Bronze"
print(tier)
```

```
Silver
```

...

(One misplaced `elif` and Tobi demotes every VIP.)

Predict: exactly at curfew

The curfew is 22:00. An order comes in at **exactly** 22:00. What prints?

```
delivery_hour = 22
print(delivery_hour < 22)
```

a) `False` b) `True` c) `Error`

...

Predict first. Commit to an answer before the next slide.

Answer: `22 < 22`

a) **`False`**: `<` is **strict**. 22 is not *less than* 22, so at 22:00 sharp the kitchen is already closed. Use `<=` when the boundary should count.

```
delivery_hour = 22
print(delivery_hour < 22)
```

```
False
```

Your turn — 10 minutes

Open the exercise (scan the QR or type the link):

beyondsimulations.github.io/Introduction-to-Python/notebooks/ex_02_a/

First **predict** what happens, then run it.

Doing Things Many Times

The copy-paste pain

Tobi totals the day's orders by hand, one `print` per order:

```
print("Falafel Wrap")
print("Pad Thai")
print("Miso Ramen")
```

```
Falafel Wrap
Pad Thai
Miso Ramen
```

. . .

Three lines for three items. A hundred orders? A hundred lines. There is a better way.

`for`: one step for every item

A `for` **loop** runs its block once for each item in a list:

```
menu = ["Falafel Wrap", "Pad Thai", "Miso Ramen"]
for item in menu:
    print(item)
```

```
Falafel Wrap
Pad Thai
Miso Ramen
```

...

`item` is the **loop variable**. It takes each value in turn, and the name is yours to choose.

The accumulator pattern

Start a total at 0, then **add each item** as the loop visits it:

```
prices = [5.00, 3.50, 6.50]
total = 0
for price in prices:
    total = total + price
print(total)
```

```
15.0
```

...

After the loop, `total` holds the sum. This is how you replace Tobi's hand-counted dashboard.

`range()`: count without a list

`range(n)` counts from `0` up to, but **not including**, `n`:

```
for i in range(3):
    print(i)      # ping the courier's phone 3 times
```

```
0
1
2
```

...

Three passes: `0`, `1`, `2`. (It stops *before* 3, a beginner's favorite surprise.)

`range()` with two and three arguments

- `range(start, stop)`: begin at `start`, stop before `stop`
- `range(start, stop, step)`: jump by `step` each time

```
print(list(range(2, 5)))       # list() shows the range as a list: 2, 3, 4
print(list(range(0, 10, 3)))   # every 3rd order gets a flyer: 0, 3, 6, 9
```

```
[2, 3, 4]
[0, 3, 6, 9]
```

Looping over a string

A string is a sequence too, and a `for` loop walks it character by character:

```
for letter in "Pad":
    print(letter)
```

```
P
a
d
```

A loop in one line (a preview)

A **list comprehension** is a `for` loop compressed into one line that builds a list:

```
doubled = [n * 2 for n in [4, 5, 6]]
print(doubled)
```

```
[8, 10, 12]
```

. . .

> **i** Note
>
> Just a taste. Session IV gives comprehensions their proper treatment. For now: recognize the shape.

Predict: where does `print` run?

The `print` is **not** indented. What does this show?

```
total = 0
for n in [10, 20]:
    total = total + n
print(total)
```

a) `10` b) `10` then `30` c) `30`

. . .

Predict first, then turn the page.

Answer: the unindented `print`

c) `30`: `print` sits outside the loop, so it runs once, after the loop finishes, showing the final total. Indent it and it would print on every pass. Indentation decides what repeats.

```
total = 0
for n in [10, 20]:
    total = total + n
print(total)
```

```
30
```

Your turn — 10 minutes

Open the exercise (scan the QR or type the link):

beyondsimulations.github.io/Introduction-to-Python/notebooks/ex_02_b/

First **predict** what happens, then run it.

Repeat Until, and Clean Text

`while`: repeat as long as

A `while` **loop** repeats **as long as** its condition stays `True`. Suppose MunchCorp undercuts you. Cut your price each round until you drop below their price:

```
price = 10.00
rounds = 0
while price >= 7.00:
    price = round(price * 0.8, 2)   # 20% off each round
    rounds = rounds + 1
print(rounds, price)
```

```
2 6.4
```

...

Two rounds and you're under 7.00. (The lab's price war against MunchCorp is your boss fight after the break.)

The loop must move toward its goal

Every pass must nudge the condition **closer to** `False`: here the price shrinks, so the loop is guaranteed to end.

...

If nothing changes, the condition never flips and the loop runs **forever**:

```
# ⚠️ NEVER run this. It never stops, and freezes the browser tab
count = 1
while count > 0:
    count = count + 1    # count only grows, so the condition stays True forever
```

...

> ⚠️ Warning
>
> An endless `while` will freeze your marimo tab. Always check that **something inside the loop changes** the condition.

`break`: an early exit

`break` jumps out of a loop immediately, handy with `while True`:

```
count = 0
while True:
    count = count + 1
    if count == 3:
        break
print(count)
```

```
3
```

...

The loop would run forever, but `break` stops it the moment `count` hits 3.

Cleaning up text

Tobi typed the daily special IN ALL CAPS with stray spaces. Strings have **methods** that return a cleaned-up **copy**:

```
raw = "  miso ramen  "
print(raw.strip())          # drop outer spaces
print(raw.strip().title())  # then Capitalise Each Word
print("special".upper())    # SHOUT
print("MOIN".lower())       # whisper
```

```
miso ramen
Miso Ramen
SPECIAL
moin
```

...

Methods can be **chained** left to right: `raw.strip().title()`.

Trimming specific characters

`.strip()` drops spaces; `.rstrip("!")` drops trailing `!`. Chain them to fix Tobi's shouting:

```
print("SALE!!!".rstrip("!"))            # SALE
print("SALE!!!".rstrip("!").title())    # Sale
```

```
SALE
Sale
```

...

The original string is never changed. Each method hands back a **new** one.

Predict: chained methods

What does this print?

```
print("  MOIN  ".strip().lower())
```

a) `MOIN` b) `moin` c) `moin`

...

Predict first, then reveal.

Answer: `"  MOIN  ".strip().lower()`

b) **`moin`**: `.strip()` removes the outer spaces, then `.lower()` lowercases the result. Chained methods run left to right, each acting on the previous one's output.

```
print("  MOIN  ".strip().lower())
```

```
moin
```

Your turn — 10 minutes

Open the exercise (scan the QR or type the link):

beyondsimulations.github.io/Introduction-to-Python/notebooks/ex_02_c/

First **predict** what happens, then run it.

To the Lab

After the break: the lab

- Head to the lab notebook: Episode 2 — The Curfew
- You'll enforce the 22:00 curfew, total the day's orders in a loop, de-shout Tobi's menu, and fight the **MunchCorp price war** with a `while` loop
- It runs entirely in your browser: no setup, just click and code

. . .

> ! Important
>
> **Download your `.py` before you leave.** Closing the tab without downloading loses your work, and downloading is exactly how you hand in the checkpoints.

Wrap-up

Three things to remember

1. `if` / `elif` / `else` run the first true branch: test the strictest condition first
2. `for` repeats over every item (accumulator: start at 0, add each); `while` repeats until its condition flips, and something inside must move it there
3. **String methods** (`.strip()`, `.title()`, `.upper()`, `.rstrip("!")`) return a cleaned copy, and chain left to right

...

> **Note**
>
> **Next time — Episode 3** starts with Checkpoint 1: 40 minutes, no AI, everything from Episodes 1–2, so keep this notebook and the last one close. Then: Tobi has pasted the same receipt code 14 times, and one small change now takes him an afternoon. We bring in functions to the rescue.

Literature

Books to start with

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. Link to free online version
- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.

...

> **Note**
>
> Nothing new here, but these are still great books to start with!

...

For more, see the literature list of this course.